CS4475 Fall 2026
Jacob Ordonez
Jjordonez@miners.utep.edu

Lab Report
Submitted on 10/14/2026

PA2: Priority-based Scheduler for xv6

Task 1. (15 pts) Implement the getprocs() system call and test it using the provided ps command.

I added the getprocs() system call so the provided ps program could display information about active processes. It reads each process's PID, state, memory size, parent PID, and name. It uses locks while reading the information and copyout() to send it to the user program. It returns the number of processes copied, or -1 if an error occurs.

I updated kernel/pstat.h, kernel/proc.h, kernel/proc.c, kernel/defs.h, kernel/sysproc.c, kernel/syscall.h, kernel/syscall.c, user/user.h, user/usys.pl, and the Makefile. I also added user/ps.c, kept the rusage structure from PA1, and changed %l to %lu to fix the memory-size printing error.

I tested it by running ps, which successfully displayed the process information shown below.

```
jacob-ordonez@jacob-ordonez-QEMU-Virtual-Machine:~/xv6-riscv$ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 1 -n
ographic -global virtio-mmio.force-legacy=false -drive file=fs.img,if=none,format=ra
w,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

init: starting sh
$ ps
pid     state           size    priority        ppid    name
1       sleeping        16384   0               0       init
2       sleeping        20480   0               1       sh
3       running         16384   0               2       ps
$
```

Task 2. (15 pts) Modify the provided ps command to print the priority of each process.

I added a priority field to each process, initialized it to 0, and made child processes inherit their parent's priority. I implemented getpriority() and setpriority(), allowing priorities from 0 to 49. I also updated getprocs() to return the priority and modified ps to display it.

I tested these changes using prioritytest. Valid priorities were accepted, invalid values were rejected without changing the priority, and the child inherited priority 10. The ps output also showed the correct priorities, as shown below.

```
$ prioritytest
Initial priority: 0
Set 49 returned: 0
Current priority: 49
Set -1 returned: -1
After invalid value: 49
Set 50 returned: -1
After invalid value: 49
Set 10 returned: 0
Child inherited priority: 10
pid     state           size    priority        ppid    name
1       sleeping        16384   0               0       init
2       sleeping        20480   0               1       sh
4       sleeping        16384   10              2       prioritytest
5       running         16384   10              4       ps
$
```

Task 3. (15 pts) Add a readytime field to struct proc, initialize it correctly, and modify ps to print a process's age.

I added readytime to kernel/proc.h and kernel/pstat.h and updated kernel/proc.c to record the current tick whenever a process becomes runnable. I also updated getprocs() to return readytime and ps to calculate age as the current tick minus readytime. Age is displayed only for runnable processes.

I tested this using matmul 5 & followed by ps. The output showed matmul as runnable with age 0, while sleeping and running processes had blank ages.

```
$ matmul 5 &
$ ps
pid     state           size    age     priority        ppid    name
1       sleeping        16384           0               0       init
2       sleeping        20480           0               1       sh
6       running         16384           0               2       ps
5       runnable        12013568        0       0               1       matmul
$ Result check: 6000
Time: 125 ticks
```

Task 4. (15 pts) Implement a priority-based scheduler.
I added constants in kernel/param.h to choose between round-robin and priority scheduling. In kernel/proc.c, I updated scheduler() to select the runnable process with the highest priority while keeping the original round-robin option.
I added user/schedtest.c and included it in the Makefile. The test uses a pipe to release two children with priorities 10 and 30. With priority scheduling, 30 ran first. With round-robin, 10 ran first in this test.
I learned how priorities affect which process runs next. To make the comparison fair, I used the pipe to prevent either child from printing before both were created. I committed and pushed the changes on the pa2 branch.

```
xv6 kernel is booting

init: starting sh
$ schedtest
Releasing children with priorities 10 and 30
Running child with priority 30
Running child with priority 10
$
```

Task 5. (15 pts) Add aging to your priority-based scheduler.
I added aging to the priority scheduler in kernel/proc.c. A runnable process gains one effective priority point for each tick it waits, up to 99, while its base priority stays unchanged. I added the aging settings to kernel/param.h and created user/agingtest.c, registering it in the Makefile.
The test runs two children with priorities 10 and 12, giving the lower-priority child a shorter job. Without aging, average response time was 2.0 ticks and average turnaround time was 4.0 ticks. With aging, these decreased to 1.0 and 3.5 ticks. In this test, aging helped the lower-priority child start and finish sooner. I restored aging to enabled after testing.

```
xv6 kernel is booting

init: starting sh
$ agingtest
priority        response        turnaround (ticks)
10              2               2
12              1               5
Average response: 1.5 ticks
Average turnaround: 3.5 ticks
$
```

```
xv6 kernel is booting

init: starting sh
$ agingtest
priority        response        turnaround (ticks)
12              0               4
10              4               4
Average response: 2.0 ticks
Average turnaround: 4.0 ticks
$
```

Task 6. (5 pts) Summary

This assignment helped me understand how xv6 keeps track of processes and chooses which one runs next. I learned how to add system calls, assign process priorities, and track how long a process waits to run. Testing aging showed me how increasing a waiting process's effective priority can help it run sooner.

One difficulty was keeping track of which files needed changes and how they connected. Working through one file at a time and testing each part helped me follow the code. I also encountered a printf error in ps, which I fixed by changing %l to %lu. Comparing the terminal results made the scheduling behavior easier to understand.

Extra Credit Task (10 points).

I used ChatGPT for step-by-step guidance, code generation and explanations, debugging, test design and interpreting results. This included getprocs(), process priorities, runnable-time tracking, priority scheduling, aging, and the test programs. I applied the changes in my xv6 repository and ran the tests shown in this report.